

NETWALK

CHOUKRAD yussef

INTRODUCTION

Netwalk est un puzzle logique dont l'objectif est de relier tous les terminaux d'une grille à un serveur central en faisant pivoter des segments de câbles.

Le joueur doit manipuler les connexions pour former un réseau unique et continu, garantissant qu'aucun composant ne reste isolé.

PRÉSENTATION GÉNÉRALE

1 -CRÉATION DES CLASSES

Notre jeu est composé d'une multitude de tuiles, on peut définir notre grille comme étant une matrice de Tuiles.

Coder la classe Tuiles est alors essentielle afin de pouvoir manipuler la grille de jeux plus tard , elle ne contient pas de « main » mais seulement des méthodes qui seront réutilisé dans les classes « main ».

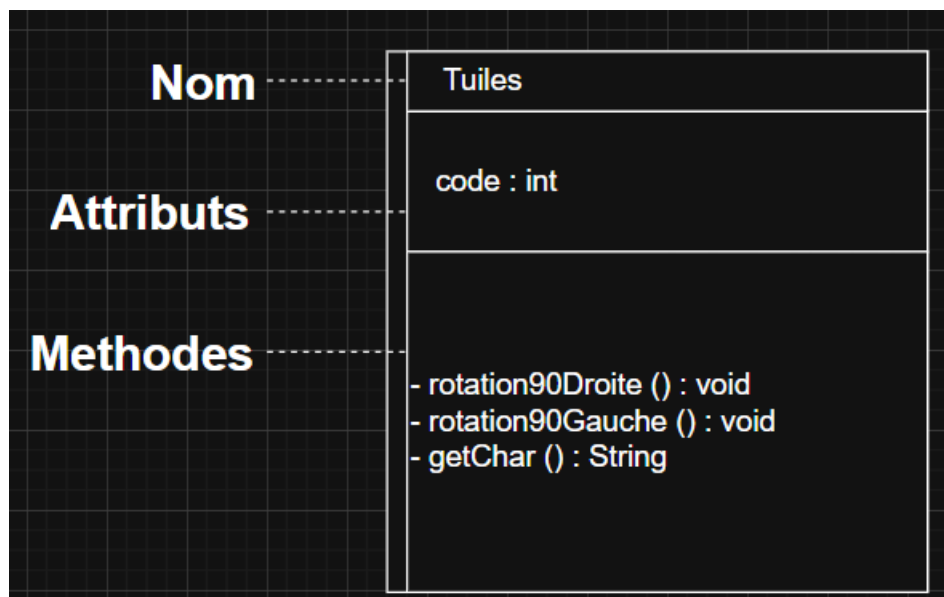


figure1- Diagramme UML Classe Tuiles

Pour des raison d'organisation et de lisibilité , j'ai créé deux « main », un pour chaque partie.

Ma premier Classe Main s'appelle Grid et ne contient pas de méthodes, mais reprend celle de tuile et affiche grâce à la bibliothèque « Scanner » mes commandes sur la console d'affichage.

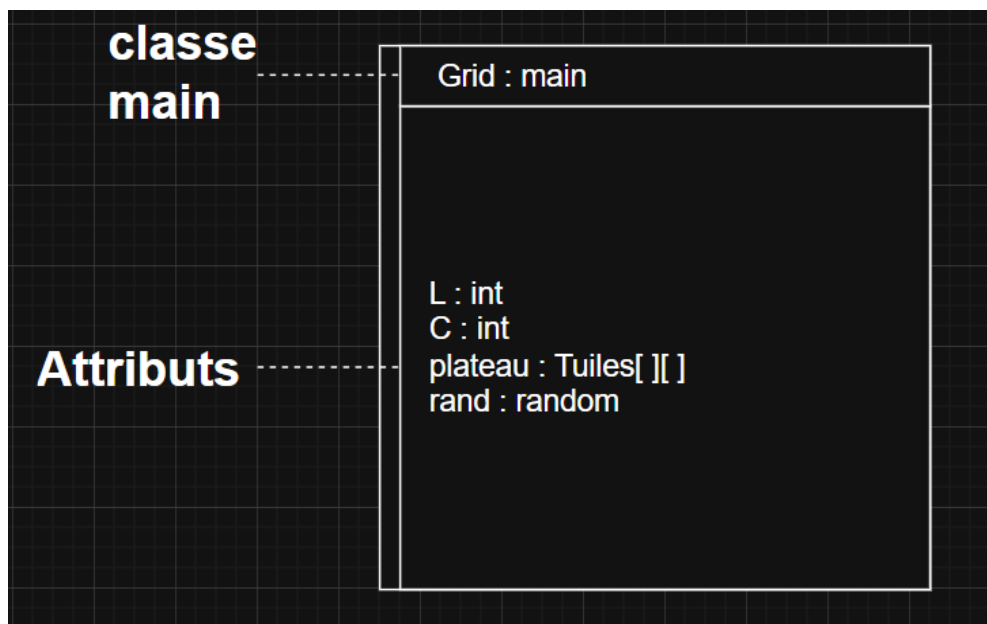


figure2- Diagramme UML Classe Grid

Ma 2eme classe Main contient la partie 2 du projet et s'appuie lui aussi sur les méthodes de ma classe Tuiles

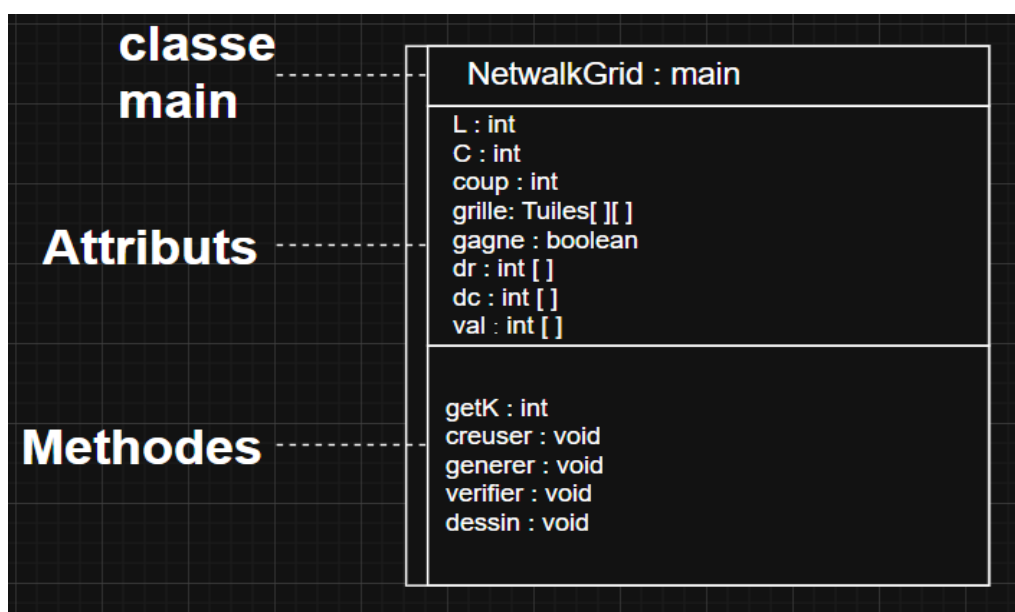


figure3- Diagramme UML Classe Grid

2 - DÉTAIL DES MÉTHODES CLASSE TUILE

On applique un raisonnement binaire pour nos méthodes de rotation, java fait automatiquement la conversion donc pas besoin d'avoir recours a une bibliothèque.

– Rotation tuile 90° vers la GAUCHE

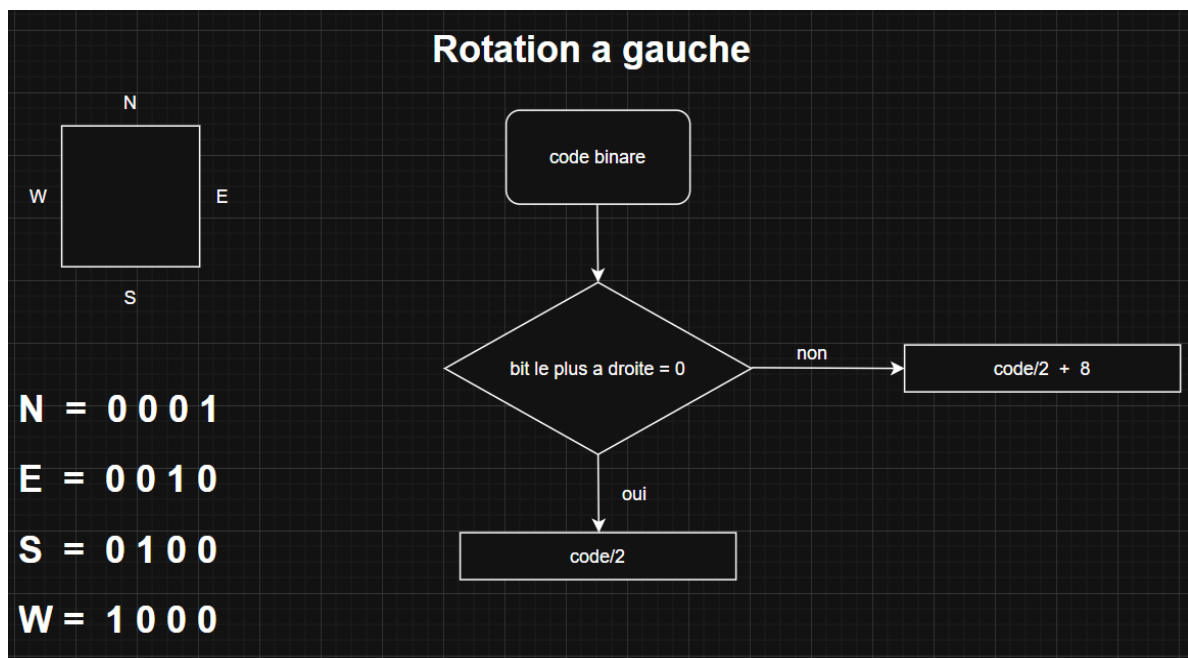


figure4 – Algorithme rotation a gauche de la tuile

– Rotation tuile 90° vers la DROITE

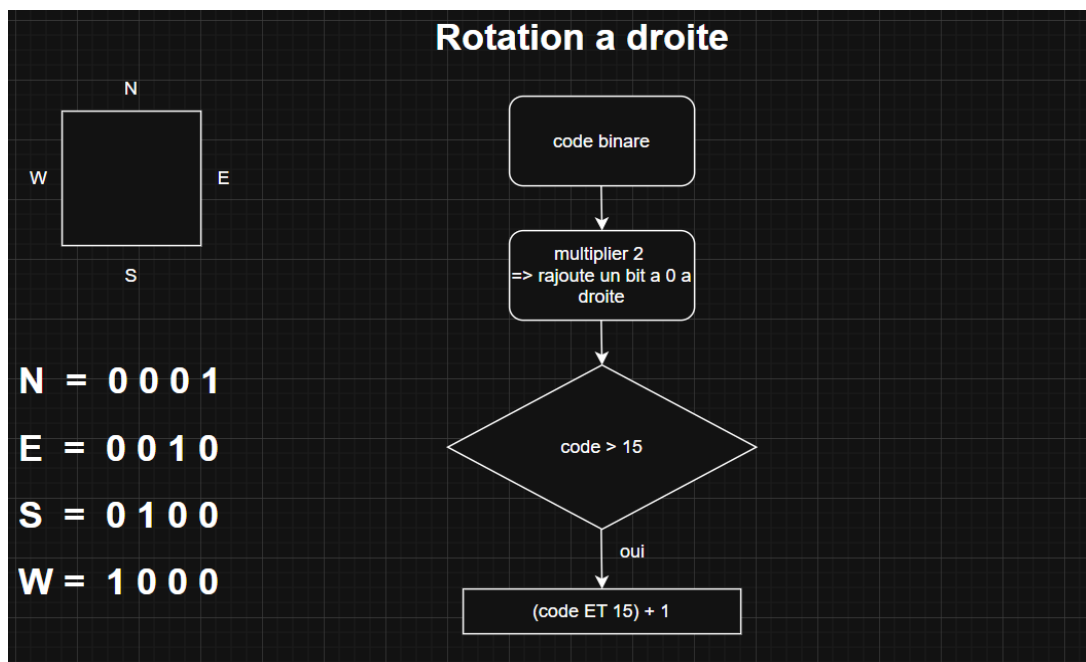


figure5 - Algorithme rotation a gauche de la tuile

– getChar

Méthode consistante à affecter à chaque caractère Netwalk un numéro entre 1 et 15.

Ce numéro est ensuite réutilisé lors de la création d'une grille aléatoire.

3- CRÉATION GRILLE DE JEUX

Lors de mes premières phases de test, j'ai rencontré un problème qui m'as pris un peu de temps.

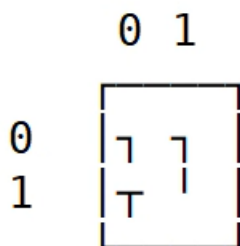
Lorsque je faisais tourner mes tuiles certaines fois j'obtenais le caractère souhaité, mais la plupart du temps ce caractère était aléatoire.

Cette erreur venait de ma méthode getChar, lorsque j'ai attribué les numéros à mes différents caractères, je les avais repris dans l'ordre propose dans l'énoncé du projet.

Or chaque caractère doit être attribue en fonction de son nombre, en prenant comme référence les codes binaires attribué à chaque orientation de la tuile.

En prenant la grille ci-dessous, vérifions si nos méthodes marchent bien en prenant la tuile (0,0)

La tuile (0,0) a pour code binaire : 1100



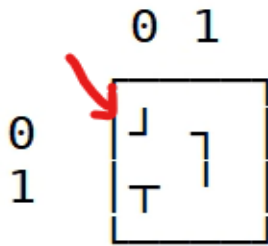
Appliquons l'algorithme pour la rotation a droite, et vérifions si cela correspond au résultat obtenu.

$$1100 * 2 = 11000$$

$$11000 = 20 > 15$$

$$11000 \& 1111 + 1 = 1001$$

Affichage code :



On remarque que le code de la tuile (0,0) est bien 1001, donc la rotation à droite marche

Par raisonnement analogue pour la rotation à gauche :

$$1001/2 + 8 = 1100$$

On retrouve ainsi la grille de départ

CRÉATION D'UNE PARTIE VALIDE

Créer une grille aléatoire n'est pas suffisant pour jouer au jeu, bien qu'on puisse déjà jouer au jeu au travers des commandes, il n'est pas certain que celui-ci puisse être résolu.

Afin de générer une partie valide il faut implémenter la méthode du « Parcours en Profondeur » aussi appeler DFS (Depth-First Search).

4- PARCOURS EN PROFONDEUR

Cette partie est sûrement la plus compliqué, mais avant de coder il faut déjà comprendre l'algorithme.

Le DFS applique une méthode « bourrin », en effet elle consiste à tester toutes les possibilités.

Pour ce qui est question d'algorithme, je me suis appuyé sur le dessin fourni afin de comprendre et faire mon code.

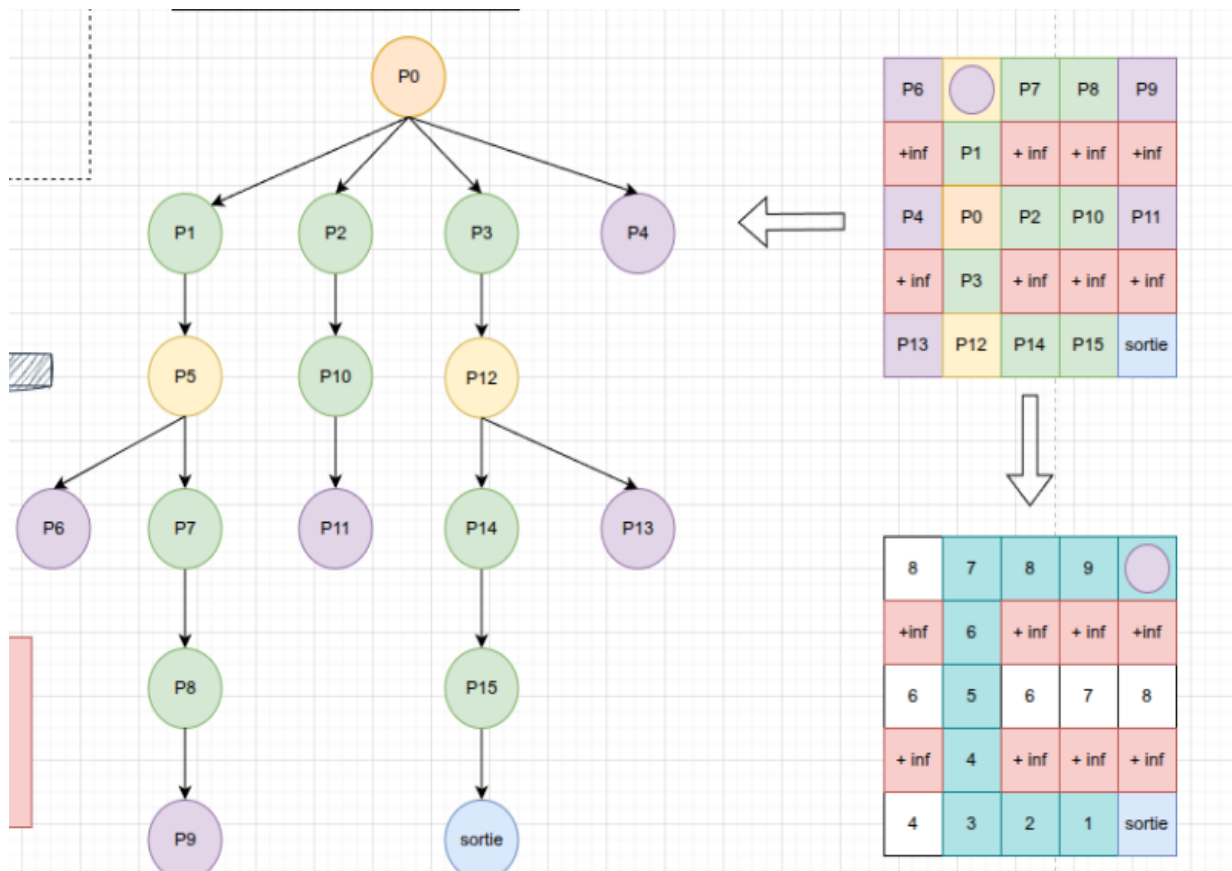


figure6 – représentation graphique DFS

On suppose qu'il existe une case qu'on a déjà visitée, puis on regarde ses voisins afin de voir si une connexion existe entre eux, puis par principe de récursivité on applique cela pour toute la grille.

Donc la case qui sera générée sera au début sera déjà reliée, puis on générera à partir de cette grille une nouvelle grille mélangée mais solvable puisque le parcours à retrouver est celui proposé avant mélange.

INTERFACE GRAPHIQUE

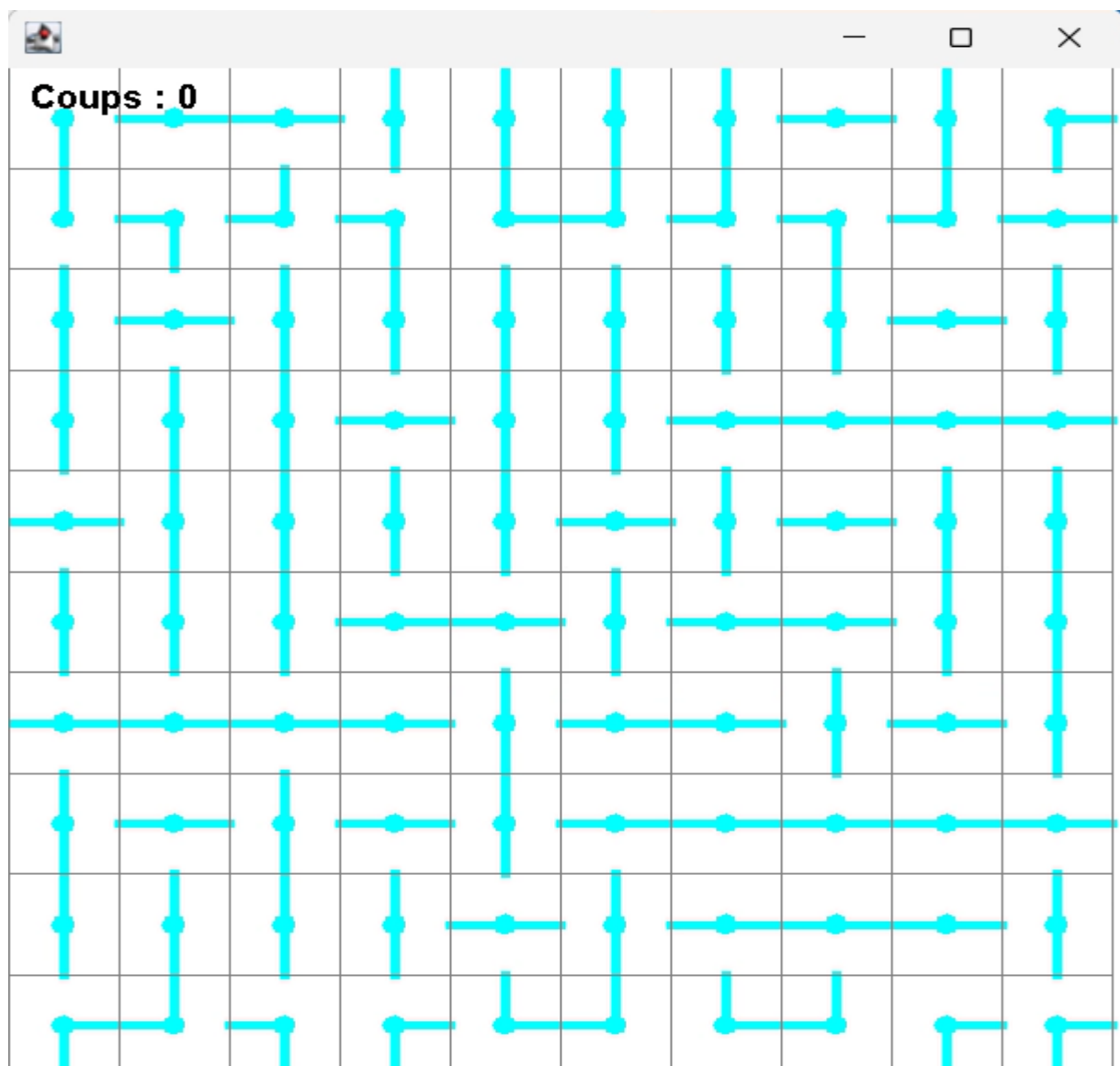


figure7 – console Netwalk 10x10

LIMITE DU CODE

Le code présente quelques limites, en effet la taille de mes tuiles restent constantes, pour une grille de taille 100X100, celle-ci sortirait de l'écran donc elle est impossible à implémenter.

Plus la dimension de ma grille est grande, plus la fenêtre graphique est grande.

J'aurais aussi aimé faire résoudre le jeu à l'ordinateur.